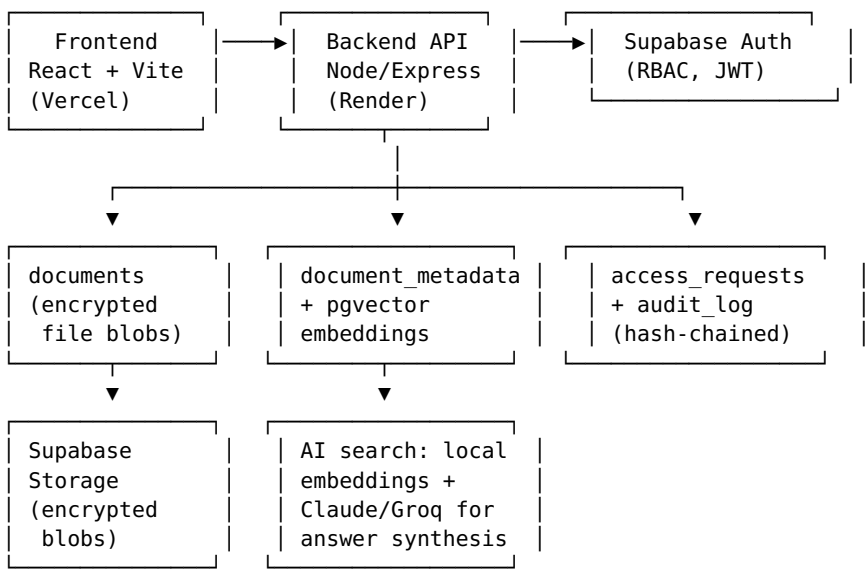


Secure Case File System - Technical Build Guide

Secure Case File System — Technical Build & Presentation Guide

Scope: A working web prototype demonstrating encrypted document storage, tamper-evident audit logging, role-based + escalated access control, digital signatures, and AI-powered natural language search over fake/sample case files — built entirely on free tools for SIH.

1. Architecture Overview



Everything above runs on free tiers. Total moving parts: 1 Postgres database (Supabase), 1 backend service (Render/Railway), 1 frontend (Vercel), 1 LLM API (Claude trial credits or Groq free tier).

2. Free Tool Stack (exact choices, with reasons)

Layer	Tool	Free tier not
Frontend framework	React + Vite + Tailwind CSS	Fully free, fast loop
Hosting (frontend)	Vercel	Free tier, auto-deploys from GitHub
Backend	Node.js + Express (or FastAPI if your team)	Free, huge libr support

	prefers Python)	
Hosting (backend)	Render.com free tier or Railway free tier	Both sleep on inactivity on free tier — mention this to judges as the only real limitation, and wake the demo server ~2 min before presentation
Database	Supabase (hosted Postgres)	Free tier: 500MB DB, more than enough for a prototype
File storage	Supabase Storage	Free tier: 1GB storage, stores your encrypted blobs
Auth + roles	Supabase Auth	Free, built-in JWT + row-level security, saves building login from scratch
Symmetric/asymmetric encryption	libsodium-wrappers (npm)	Free, audited, simpler API than raw OpenSSL
Vector search	pgvector (Postgres extension, included free in Supabase)	Avoids paying for Pinecone/Weaviate — one DB does everything
Embeddings model	all-MiniLM-L6-v2 via @xenova/transformers (runs in Node, no API key/cost)	Free, runs locally, no rate limits during demo
LLM for answer synthesis	Claude API (trial credits) with Groq (free, fast Llama models) as backup	Groq protects you if you burn through Claude credits during testing
OCR	Tesseract.js	Free, runs in-browser or in Node
Digital signatures	jsrsasign (JS) or Node's built-in crypto module (RSA-SHA256 sign/verify)	Free, no external dependency needed
Hash-chained audit log	Plain Postgres table + Node's built-in <code>crypto.createHash('sha256')</code>	No blockchain infra required
Version control / collaboration	GitHub (free, private repos for teams)	Standard

One sentence for judges: “The entire system runs on free tiers — Supabase for database, storage, and auth, Groq/Claude for AI, Vercel and Render for hosting — so it’s a fully working, zero-cost, live-deployable prototype, not just a local demo.”

3. Database Schema

-- Users & roles (Supabase Auth handles the auth table; this extends

```

it)
    create table profiles (
        id uuid references auth.users primary key,
        full_name text,
        role text check (role in ('officer', 'senior_inspector', 'judge',
'admin'))),
        case_ids uuid[] -- cases this user is assigned to
    );

-- Cases
create table cases (
    id uuid primary key default gen_random_uuid(),
    case_number text unique not null,
    crime_type text,
    status text default 'open',
    created_at timestamptz default now()
);

-- Encrypted documents (the sensitive blob)
create table documents (
    id uuid primary key default gen_random_uuid(),
    case_id uuid references cases(id),
    encrypted_file_path text not null, -- path in Supabase Storage
    encrypted_dek text not null, -- data encryption key,
wrapped per-user
    uploader_id uuid references profiles(id),
    uploaded_at timestamptz default now(),
    signature text, -- digital signature, if
signed
    signed_by uuid references profiles(id)
);

-- Metadata / "summary card" – used for AI search, less sensitive
than the raw file
create table document_metadata (
    id uuid primary key default gen_random_uuid(),
    document_id uuid references documents(id),
    case_id uuid references cases(id),
    suspect_name text,
    crime_type text,
    incident_date date,
    suspect_age int,
    location text,
    extracted_summary text,
    embedding vector(384) -- pgvector column, 384-dim for MiniLM
);

-- Access requests / escalation
create table access_requests (
    id uuid primary key default gen_random_uuid(),
    document_id uuid references documents(id),
    requester_id uuid references profiles(id),
    status text default 'pending' check (status in
('pending', 'approved', 'denied')),
    reviewer_id uuid references profiles(id),
    requested_at timestamptz default now(),
    reviewed_at timestamptz,
    expires_at timestamptz
);

-- Tamper-evident audit log (hash chain)
create table audit_log (
    id bigint generated always as identity primary key,
    document_id uuid,
    user_id uuid,
    action text, -- 'upload', 'view', 'request', 'approve', 'deny', 'sign'

```

```

    timestamp timestampz default now(),
    prev_hash text,
    hash text
  );

```

Hash-chain logic (Node, pseudocode):

```

const crypto = require('crypto');

async function logAction({ documentId, userId, action }) {
  const last = await db.query(
    `select hash from audit_log order by id desc limit 1`
  );
  const prevHash = last.rows[0]?.hash || 'GENESIS';
  const timestamp = new Date().toISOString();
  const hash = crypto
    .createHash('sha256')

    .update(`${documentId}${userId}${action}${timestamp}${prevHash}`)
    .digest('hex');

  await db.query(
    `insert into audit_log (document_id, user_id, action, timestamp,
prev_hash, hash)
  values ($1,$2,$3,$4,$5,$6)` ,
    [documentId, userId, action, timestamp, prevHash, hash]
  );
}

```

To verify integrity later (great live-demo trick): recompute each row's hash from its stored fields and confirm it matches prev_hash of the next row. If someone edited a row directly in the DB, the chain breaks visibly — show this on stage.

4. Fake Case File Data (build this first)

Generate this before writing any other code — it unblocks every other feature and makes your demo feel real.

What to generate (use Claude/ChatGPT to draft the text): - 3 fake cases, each with a case number (e.g. 2024-CR-113), crime type, and status - Per case: 1 FIR, 1-2 witness statements, 1 charge sheet, 1 forensic report - Vary the content meaningfully — include specific searchable details (a red car, a suspect's age, a location) so your AI search demo has something real to find - Convert 2-3 of these to **image screenshots** (paste text into a Word doc, screenshot as PNG) to demonstrate OCR - Save everything in a /fake-data folder in your repo with a manifest JSON:

```

[
  {
    "case_number": "2024-CR-113",
    "crime_type": "Theft",
    "documents": [
      { "file": "fir_113.pdf", "suspect_name": "Rahul Menon",
"suspect_age": 27, "location": "MG Road" }
    ]
  }
]

```

Write a small seed script (seed.js) that reads this manifest and uploads everything through your real API — this doubles as your first end-to-end test.

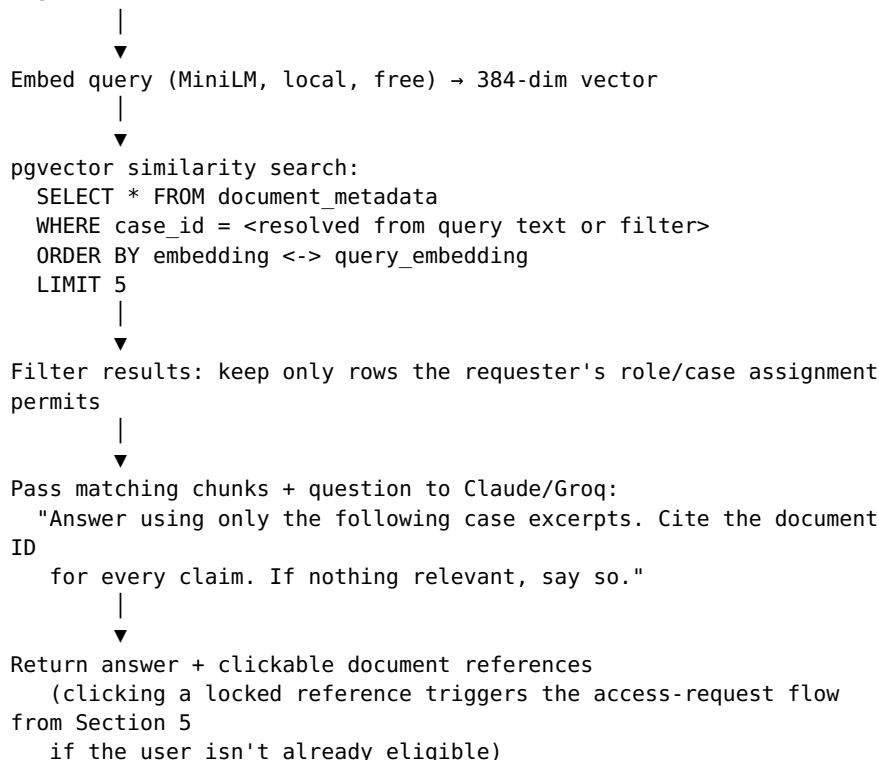
5. Build Order (in priority order — build top-down, demo works at every stage)

1. **Auth + roles** — Supabase Auth, seed 4 demo users (officer, senior_inspector, judge, admin)
2. **Encrypted upload/download** — libsodium keypair per user at signup; AES-256 DEK per file, DEK wrapped with recipient's public key
3. **Metadata extraction + storage** — on upload, pull structured fields (manually entered in a form for the prototype, or auto-extracted via a simple regex/LLM pass on OCR'd text)
4. **Audit hash-chain** — wire `logAction()` into every route (upload, view, request, approve, deny)
5. **Access request + escalation workflow** — eligibility check → auto-grant or create pending request → senior inspector approval queue
6. **Digital signatures** — sign/verify button using Node crypto RSA methods
7. **AI search (RAG)** — embed metadata + OCR'd text chunks into pgvector → natural language query → similarity search filtered by role/case → Claude/Groq synthesizes answer with citations
8. **UI polish** — dashboard, case list, audit trail timeline view, pending requests badge

Why this order: each step is independently demoable. If you run out of time, you can stop after step 5 or 6 and still have a coherent, working story — steps 7-8 are the “wow” layer on top.

6. AI Search Pipeline (the standout feature — get this right)

User types: "Show me evidence mentioning a red car in Case 2024-CR-113"



This is the sentence that ties your whole system together for judges:
“The AI can tell you a relevant document exists, but opening it still goes through the same access controls as everything else — the AI is not a backdoor.”

7. Deployment Checklist (do this a day before presenting, not the night before)

- ☐ Push final code to GitHub
 - ☐ Connect repo to Vercel (frontend) and Render/Railway (backend) — auto-deploy on push
 - ☐ Set environment variables (Supabase URL/keys, Claude/Groq API key) in both platforms’ dashboards, never commit them
 - ☐ Run the seed script against your **production** Supabase instance so the deployed demo has data
 - ☐ Test the full flow end-to-end on the deployed URL, not just localhost
 - ☐ Have a **backup screen recording** of the full demo in case venue wifi fails — this is standard practice, judges won’t mind
 - ☐ Ping the Render/Railway free-tier backend ~5 minutes before your slot so it’s not “cold” when you go on stage
-

8. How to Present It (SIH judging structure)

SIH judges typically score on: **problem understanding, technical feasibility, innovation, and impact/scalability**. Structure your pitch to hit all four explicitly.

Suggested demo flow (aim for 5-7 minutes):

1. **Problem (30 sec)** — one sentence on the real-world pain point, don’t over-explain, judges already read the problem statement
2. **Architecture slide (30 sec)** — show the diagram from Section 1, say it out loud once, move on
3. **Live demo (3-4 min), in this order:**
 - Log in as Officer → upload a fake FIR → show the file is encrypted at rest (open Supabase Storage, show gibberish bytes)
 - Search naturally: “Find witness statements mentioning a red car” → AI returns a synthesized answer with document citations
 - Click a cited document the officer isn’t authorized for → show the access-request flow trigger
 - Switch to Senior Inspector tab → see the pending request → approve it
 - Switch back to Officer tab → now the document opens
 - Open the audit trail view → show the full chain of actions, then demonstrate tamper-detection (edit a row directly in the DB, show the chain breaks)
4. **Tech stack + free-tier cost slide (30 sec)** — reinforces feasibility and scalability
5. **Future scope (30 sec)** — real KMS/HSM key management, Aadhaar-based eSign integration, Hyperledger for a production-grade ledger, mobile app for field officers

Anticipate these judge questions and have answers ready: -

“Isn’t this just a file storage app with extra steps?” → No — the access-request escalation and tamper-evident audit trail are the core innovation, not the storage itself. - “How is this different from Google Drive with permissions?” → Drive doesn’t give you cryptographic

tamper-evidence, per-file key isolation, or an AI search layer that respects access control at the retrieval stage. - *“What happens if the senior inspector is unavailable?”* → Mention this as a known gap/future feature: a delegation or timeout-based fallback approver. - *“Is this actually blockchain?”* → Be honest: it’s a hash-chain, which gives the same tamper-evidence property as a private blockchain without the infrastructure overhead — call out Hyperledger Fabric as the production upgrade path.

Slide deck order: Problem → Architecture → Live demo → Tech stack/cost → Future scope. Keep text on slides minimal — the live demo should carry most of the weight.

9. What to Explicitly Avoid Over-Building

- Real blockchain infrastructure — hash-chain + a confident verbal explanation is more defensible under questioning than a shallow blockchain integration
- Real KMS/HSM — per-user keypairs in your DB (encrypted with a master key from an env variable) is a reasonable prototype simplification, just say so out loud
- Real Aadhaar/government eSign integration — high effort, low demo payoff unless someone on the team has done it before
- Building your own auth system — Supabase Auth is free and saves days of work